

MTSC2025

中国互联网测试开发大会

TESTING SUMMIT CONFERENCE CHINA 2025

上海  
站

# 质效革新·智领未来

2025/7/12 | 上海喜来登由由大酒店 主办方: TesterHome

**MTSC2025** 中国互联网测试开发大会 上海站  
TESTING SUMMIT CONFERENCE CHINA 2025

# AI视觉模型结合页面知识图谱助力 UI自动化完成视觉需求测试落地实践

讲 师：仲思宇

质效革新·智领未来

主办方：TesterHome



## 仲思宇

58同城 – 本地生活 - 质量部 - 测试平台研发负责人

---

负责58到家测试工具体系规划与架构设计，拥有多项测试工具发明专利。深耕客户端与服务端自动化测试多年，擅长业务质量体系规划及技术赋能落地，丰富的测试平台建设与效能提升经验。

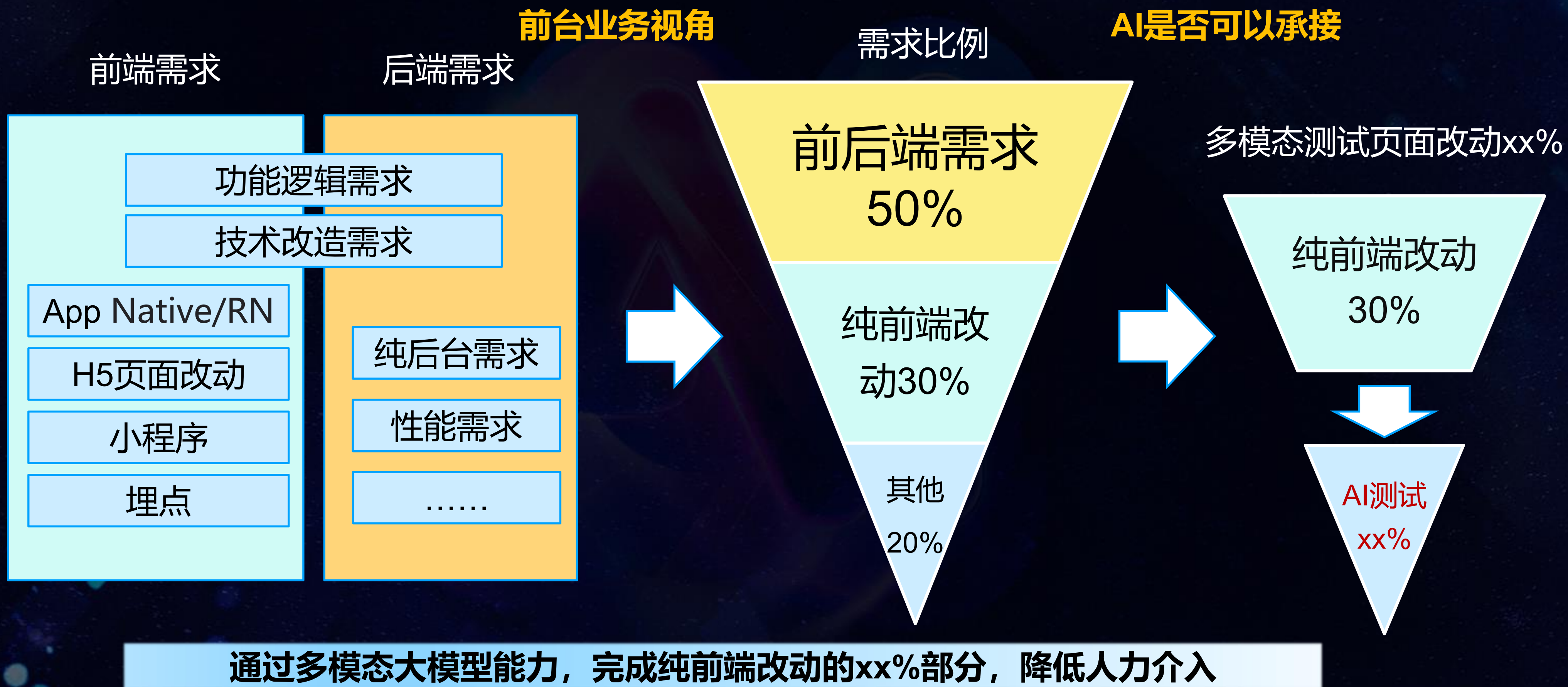
专注领域：质量体系建设、AI测试体系建设

# 目录

## CATALOGUE

- 01 背景介绍-我们能AI做哪些现有迭代类型的测试
- 02 AI视觉大模型建设UI自动化测试能力
- 03 需求文档如何生成可执行提示词
- 04 知识图谱增强知识库，提高执行链路准确率
- 05 如何自更新业务链路知识库
- 06 应用效果与展示
- 07 后续规划

# 我们能用AI做哪些现有迭代类型的测试



## 传统UI自动化问题

框架有一定学习成本

脚本编写和维护成本高

页面固化操作

无法快速适配其他业务

## UI自动化优势 + AI

## 通过AI优化后想要的

自然语言、需求描述驱动任务

不需要写固定脚本

自适应页面操作

低成本推广

**AI自适应测试，完成 xx%测试率**  
低成本部署、测试使用和扩展到其他业务等

支持的端

|         |        |    |     |
|---------|--------|----|-----|
| Android | PC-Web | H5 | 小程序 |
|---------|--------|----|-----|

支持的测试能力

|       |      |      |      |
|-------|------|------|------|
| AI端测试 | AI断言 | 回放测试 | 页面遍历 |
|-------|------|------|------|

操作端能力

|       |      |       |       |
|-------|------|-------|-------|
| 打开App | 打开网址 | 页面结构  | 断言解析  |
| 点击    | 长按   | 截图    | 返回    |
|       |      | 输入    | 滑动    |
| 录制视频  | 设备尺寸 | 滑动到底部 | ..... |

知识库

|      |      |      |      |
|------|------|------|------|
| 需求文档 | 测试用例 | 业务地图 | 页面结构 |
|------|------|------|------|

配置及链路功能

|            |         |
|------------|---------|
| 大模型配置      | 生成报告    |
| 多场景任务      | 执行日志    |
| 关联测试覆盖率    | 需求解析提示词 |
| 实时反馈设备执行过程 |         |
| 无图、单图和多图决策 |         |

工作流

|         |
|---------|
| Dify    |
| RAGFlow |

知识图谱

|       |
|-------|
| Neo4j |
|-------|

## AI视觉大模型建设UI自动化测试能力



如何让大模型以人的思路执行测试

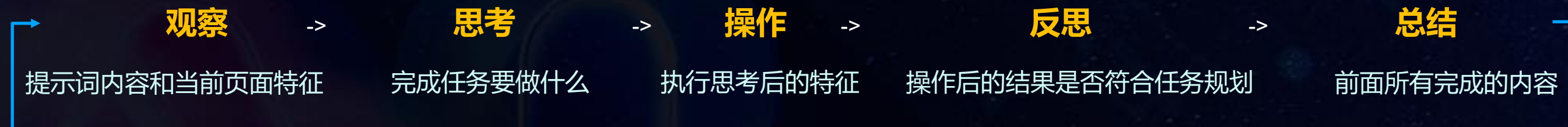


# 驱动终端的关键步骤



根据自然语言，对每一步进行观察，并执行设备  
如：在日常保洁品类下一单深度保洁订单

执行过程



观察特征“日常保洁”-> 思考,操作“日常保洁”-> 操作,点击-> 反思,“是否正确操作，错误的话返回上一页”-> 总结前面所有完成的内容

《〈每一步〉》循环上述5个环节，完成整体下单操作

提高执行准确率\_审查机制矫正错误流程

问题1：增强模型理解能力



动态prompts  
+  
审查流程



多模态观察

DeepSeek 审查结果并优化推理流程

效果对比

| #        | 接入审查   | 未接审查 |
|----------|--------|------|
| 全部操作的成功率 | 66.67% | ---  |
| 步数正确的百分比 | 70%    | 10%  |
| 决策的准确率   | 61.67% | 10%  |
| 审查的准确率   | 83.33% | ---  |

每一步之后的思考过程，确保推理步骤的正确性

A步骤



B步骤



C步骤



D步骤 .....



审查结果  
是否推进任务



场景1:

进入错误页面，返回上一层

场景2:

页面没变化

补充提示词优化策略

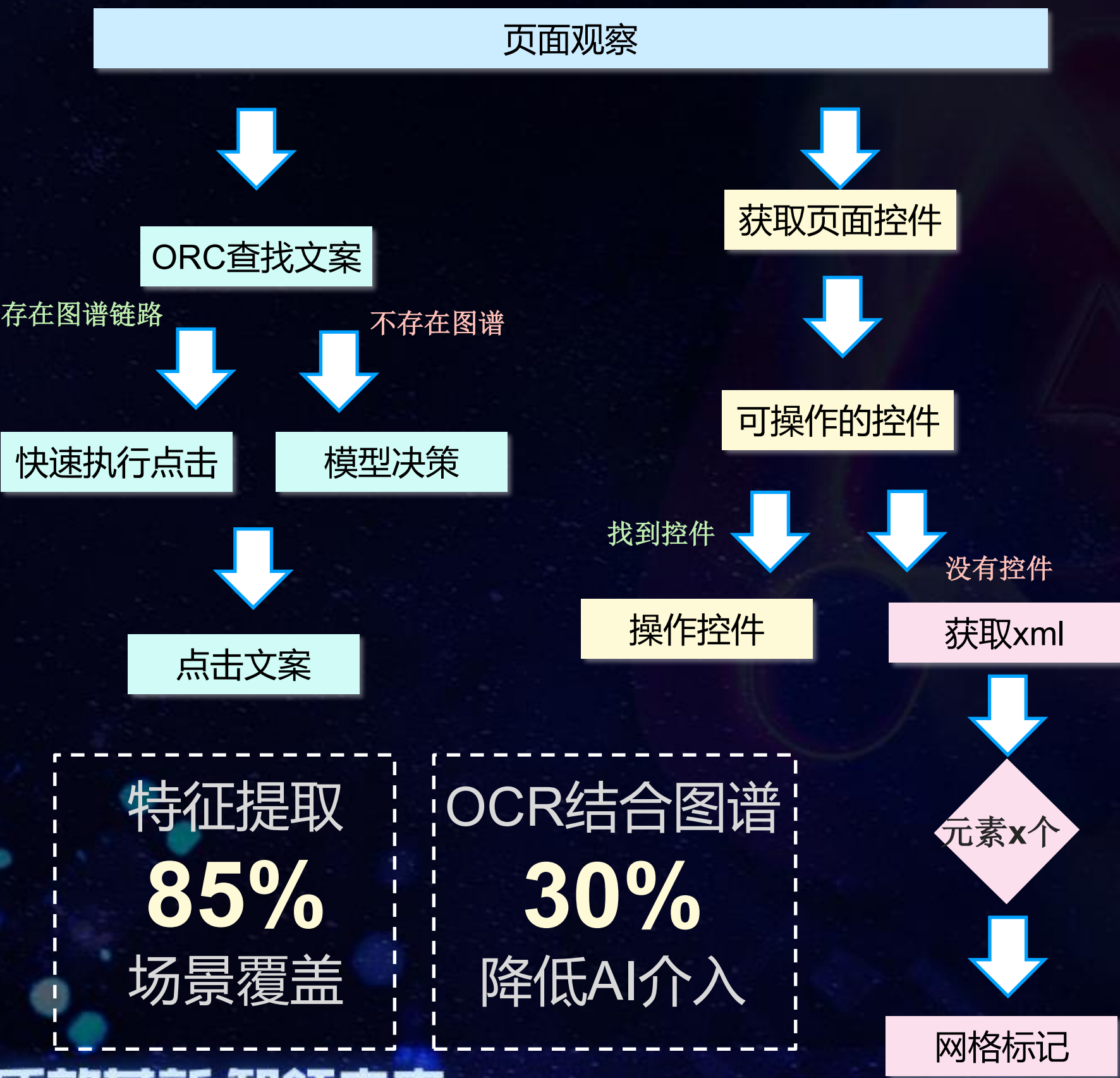
过滤无效元素

重新执行

继续任务

# 提高执行准确率\_多种方式提高元素

## 问题2：点击元素未被识别



质效革新·智领未来

### OCR标记-高



### 控件标记-中



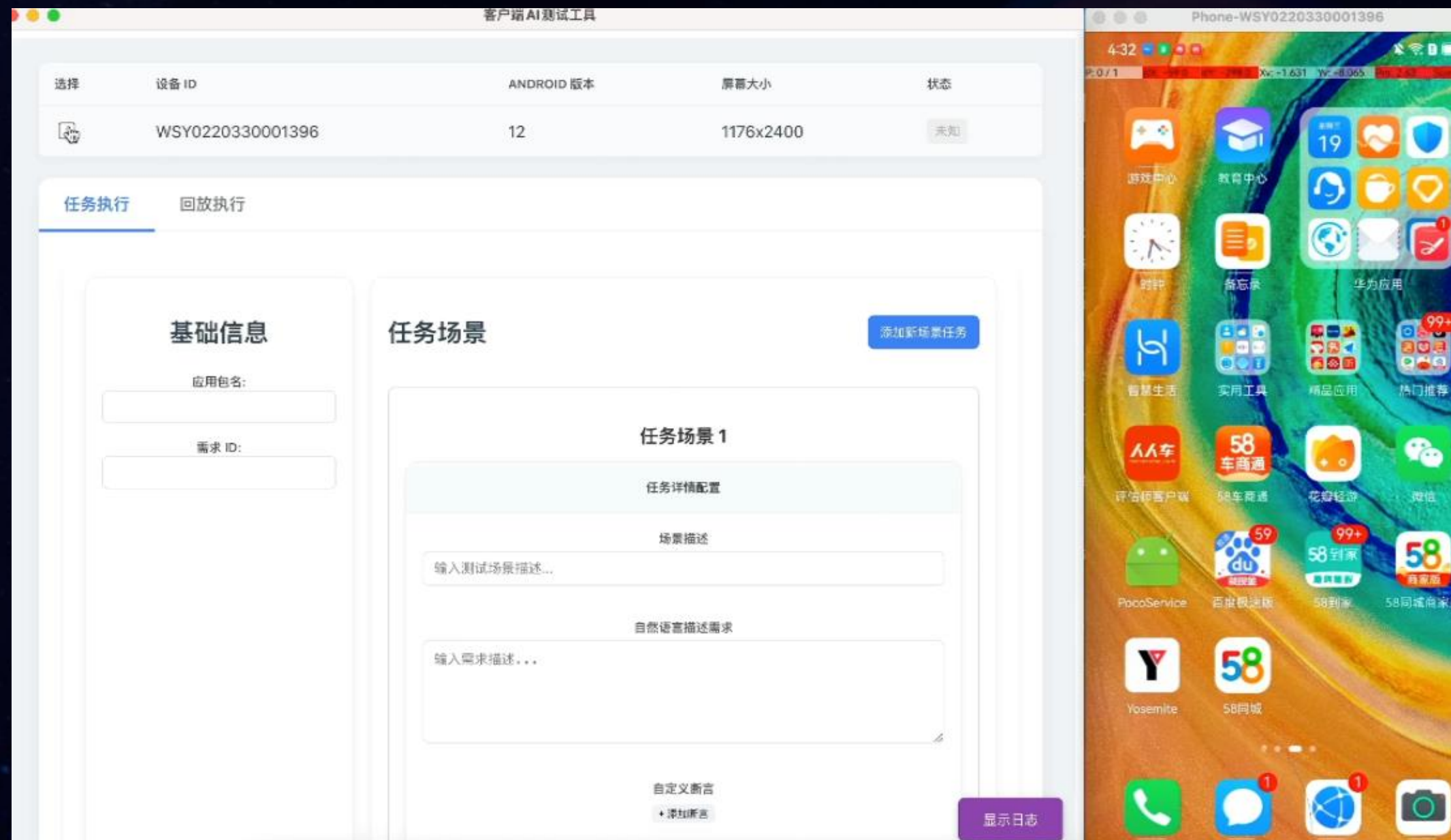
### H5兜底网格标记-低



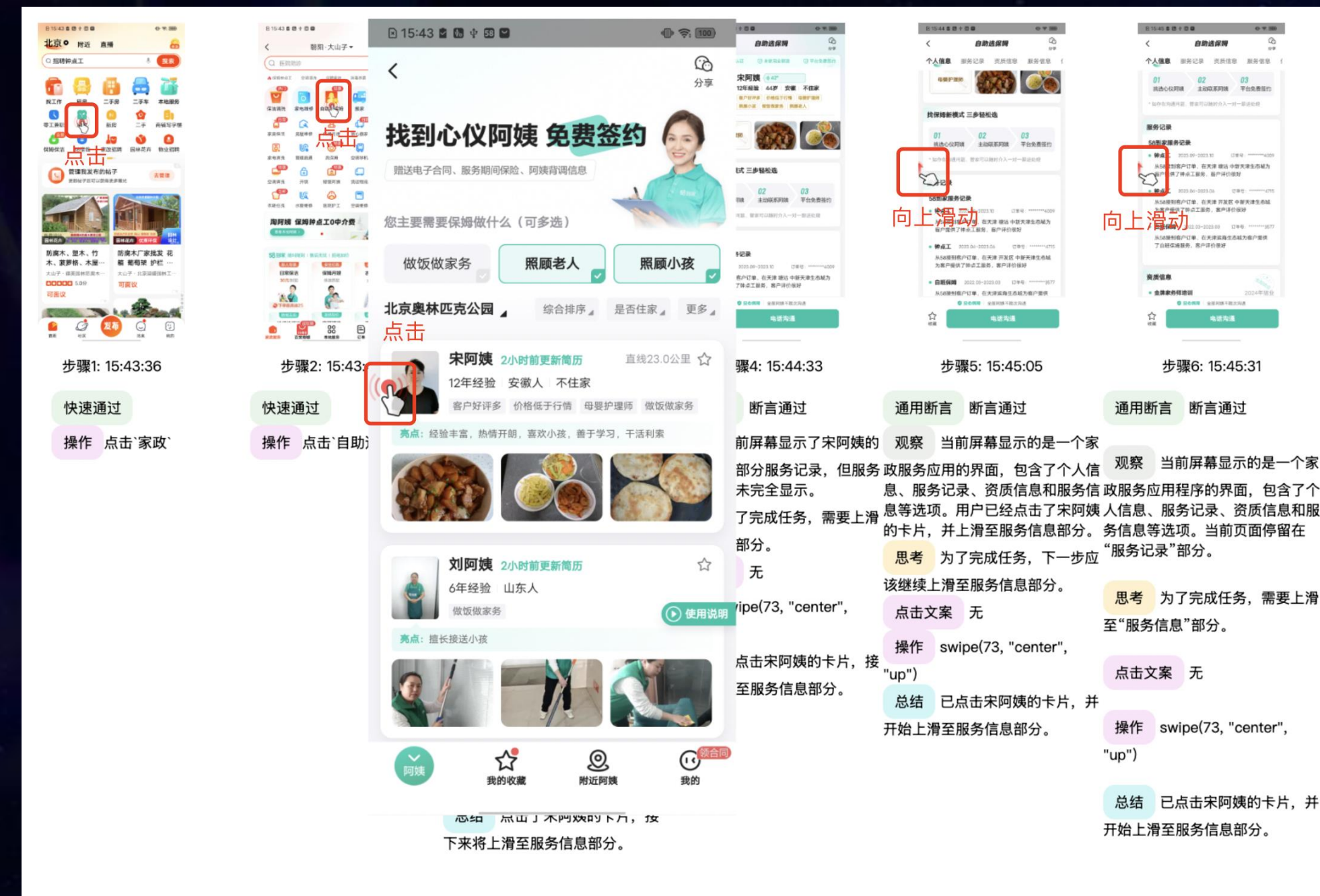
主办方：TesterHome

工具产品形态，降低使用成本

详细的测试报告信息  
每一步操作的日志  
记录大模型执行的思考过程



桌面版测试工具  
简化使用成本，不需要部署环境



## 需求文档如何生成可执行提示词

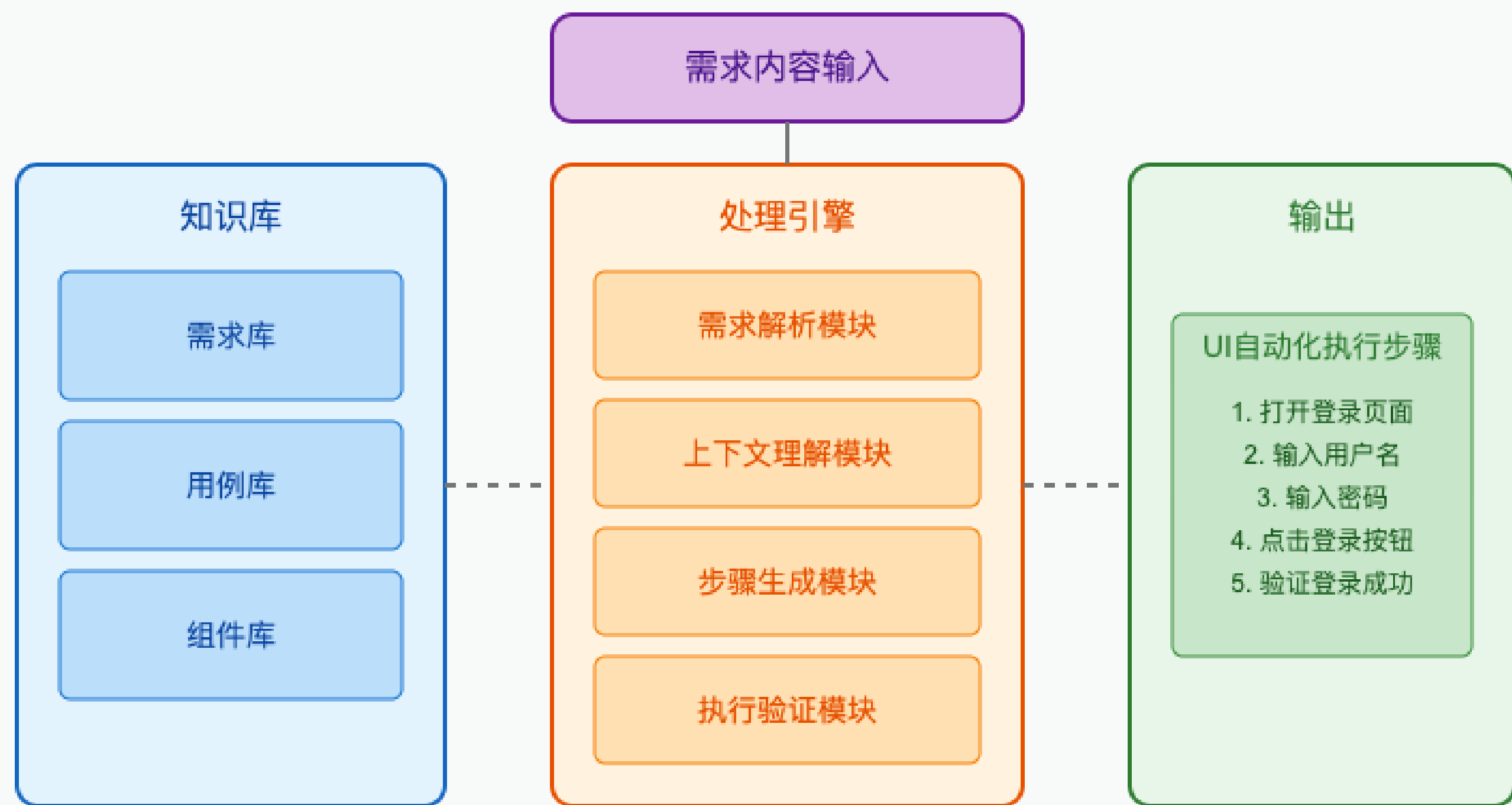
AI视觉  
大模型能力  
驱动UI自动化

通过需求文档  
生成AI可执行的  
提示词

.....

.....

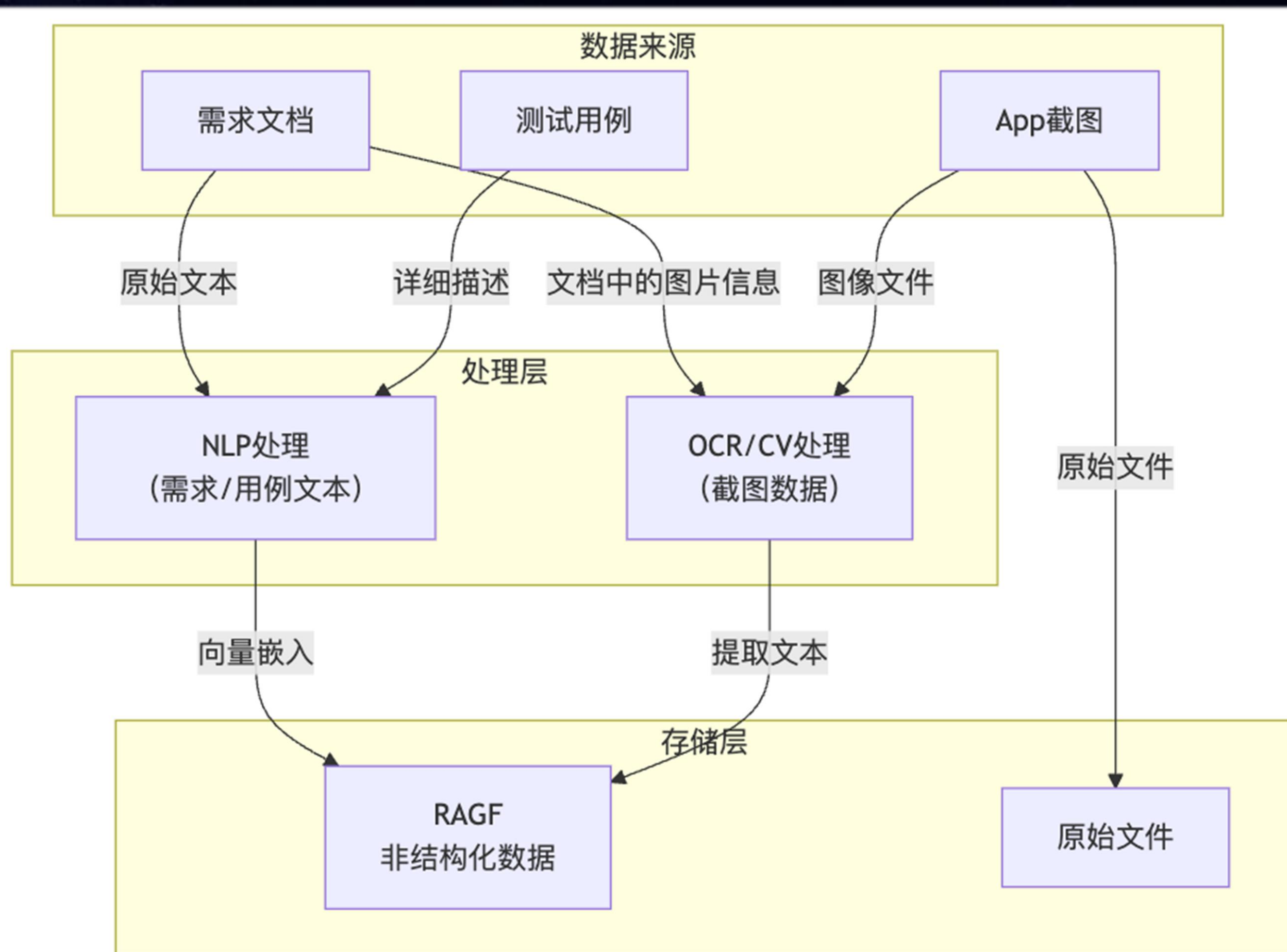
# 需求解析 + 知识库 生成客户端AI执行提示词 整体结构



把「需求→用例→页面组件」  
三种信息汇聚成可查询、  
可推理的知识库 (KB)

通过 RAG (Retrieval-Augmented-  
Generation) + LLM  
在给定需求文本时,  
自动拉取相关上下文并产出可直接落  
地的 UI 自动化步骤

# 清洗不同纬度的信息形成知识库



利用大语言模型做文本理解和摘要

- 需求文档中的业务流程、关键功能点、约束条件抽取
  - 测试用例中步骤与需求对应关系抽取
- 利用CV模型对图片组件信息做摘要
- 提取组件内容
  - 理解一组图片隐含的业务语义

根据语言生成页面执行步骤：搬家流程



## 搬家服务查询流程 ①

操作步骤：

进入本地服务页面

在初始页面点击“本地服务”图标

浏览搬家服务分类

在本地服务页面点击“搬家”分类

查看搬家店铺详情

点击具体搬家店铺（如“58到家自营搬家旗舰店”） ①

查看搬家店铺评价

点击店铺详情页的“评价”标签

完成评价查看

浏览店铺评价信息

相关图片路径：

步骤1: /opt/images/20250623143233.jpg (image\_id: 1750928659451) ①

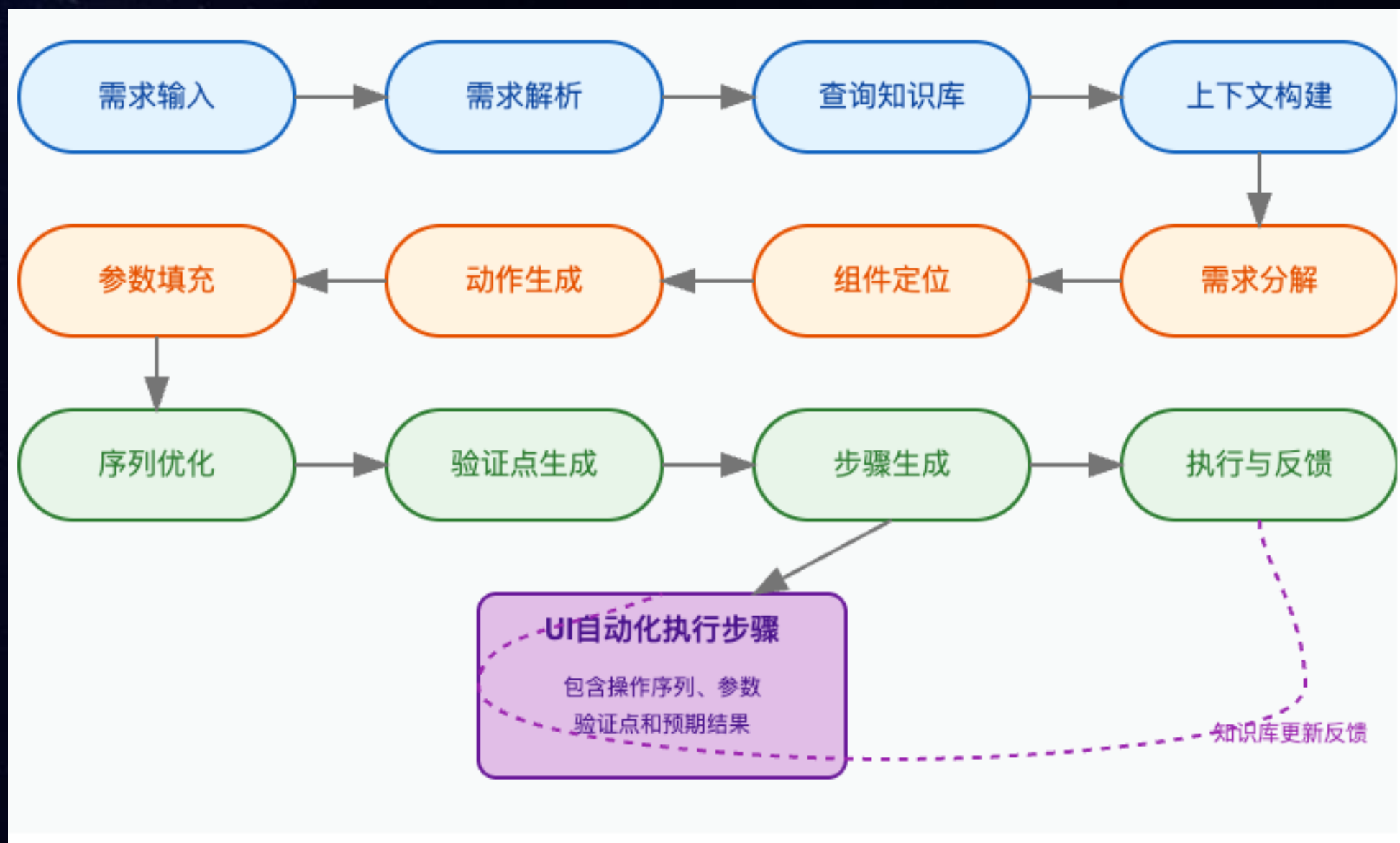
步骤2: /opt/images/20250623143233.jpg (image\_id: 1750928659451)

步骤3: /opt/images/20250623143252.jpg (image\_id: 1750928674250)

步骤4: /opt/images/20250623143303.jpg (image\_id: 1750928723220) ①

步骤5: /opt/images/20250623143325.jpg (image\_id: 1750928777420) ①

# 通过需求ID结合知识库内容形成AI可理解的提示词



## 需求处理阶段（蓝色）

- 将原始需求进行理解、关联知识库信息，并完成拆解，形成细粒度的执行项

## 动作生成阶段（橙色）

- 将拆解出的需求转化为自动化操作动作，并绑定相关组件



## 执行序列构建阶段（绿色）

- 对生成的操作序列进行优化、补充验证点并最终形成可执行的步骤

## UI自动化执行步骤输出（紫色虚线框）

- 从“步骤生成”这一阶段输出的内容将包含操作序列、参数、验证点和预期结果
- 此输出还与“知识库更新反馈”构成闭环，进一步强化系统能力

UI路径推荐

83%

覆盖率提升

提示词编写

60%

成本下降

## 知识图谱增强知识库，提高执行链路准确率

AI视觉  
大模型能力  
驱动UI自动化

通过需求文档  
生成AI可执行的  
提示词

知识内容沉淀  
图谱补强链路  
准确性

.....

# 为什么需要知识图谱增强知识库?

## 痛点1：业务链路复杂

### 自动化执行缺乏上下链路支持

#### (1) 业务链路信息不完整

- 原始知识库中的业务链路描述可能仅包含高层逻辑，缺乏与实际UI操作相关的细节。
- 导致无法精确定位UI元素或生成针对性的自动化操作。

#### (2) 上下文信息断裂

- 查询到的业务链路与实际UI操作之间存在断层，自动化脚本需要额外补充上下文信息。增加了人工干预的时间和维护成本。

#### (3) 动态场景难以处理

- 在复杂业务中，UI布局可能根据条件动态变化，仅依赖业务链路难以应对这种场景。
- 自动化脚本执行失败率高，影响效率。

## 痛点2：UI操作精度不足

### 影响自动化执行效果

#### (1) UI元素定位模糊

- 业务链路中通常没有明确的UI元素定位信息（如按钮、输入框的唯一标识）。
- 自动化执行脚本需要手动补充定位信息，影响自动化执行的精度。

#### (2) 页面结构多样性

- 不同页面的UI结构复杂且多样，仅依赖业务链路无法适配多种情况。
- 缺乏统一的UI映射逻辑，导致脚本复用性差。

#### (3) 操作路径缺乏验证

- 无法基于业务链路对UI操作路径进行验证，可能导致操作偏离预期目标。
- 增加了调试和问题排查的难度。

如何让大模型上帝视角执行测试

我们



以过往经验之舟，渡向未来确定之事

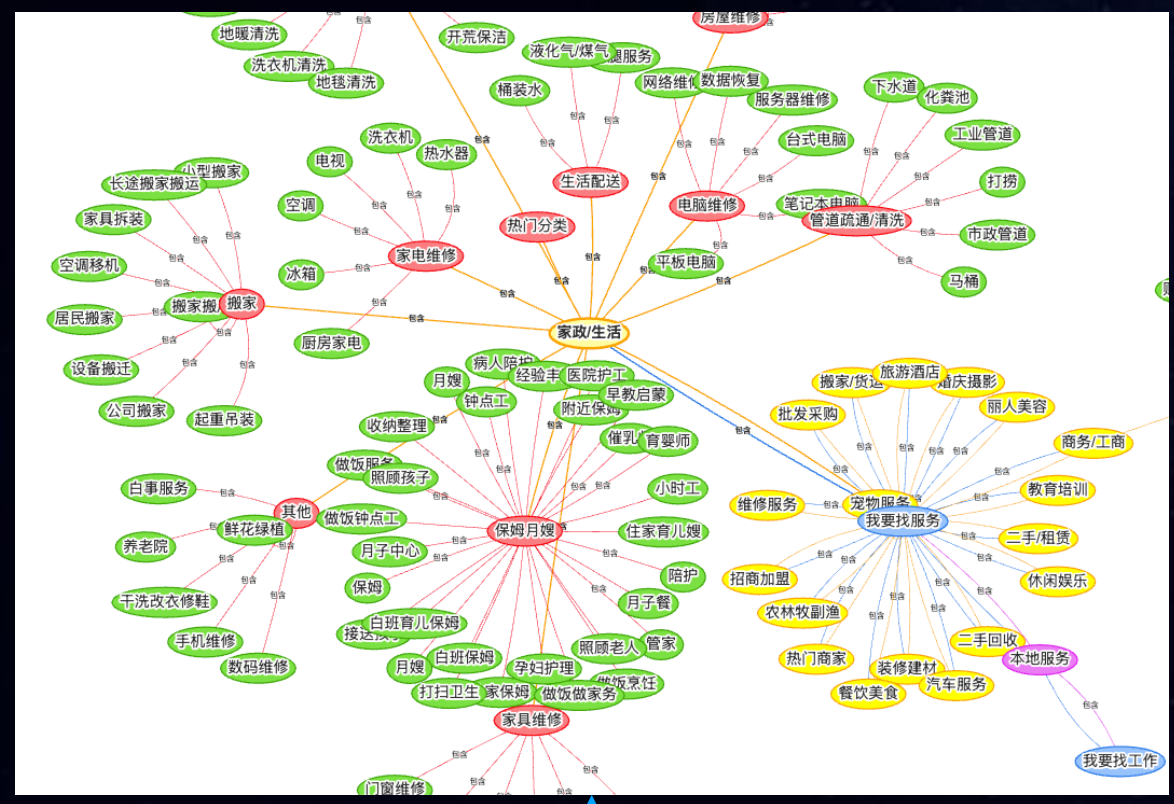
让未来需要预判，  
变成现在只需要做观察

让大模型理解App层级关系和页面布局

通过沉淀的知识普图，获取某个页面的层级链路

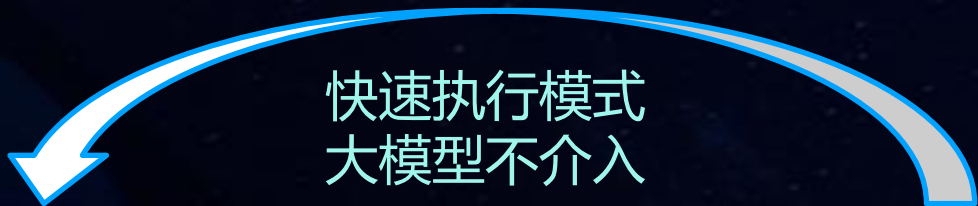
AI Agent

获取到层级链接  
A -> B -> C -> D  
在D页面做观察决定





场景：{ “query”: “进入住家保姆”}  
返回结果： “data”: “1.点击本地服务\n2.点击我要找服务\n3.点击家政/生活\n\n4.点击保姆月嫂\n5.点击住家保姆\n”



退出快速执行模式

执行失败

大模型介入

大模型分析 → 执行后续操作

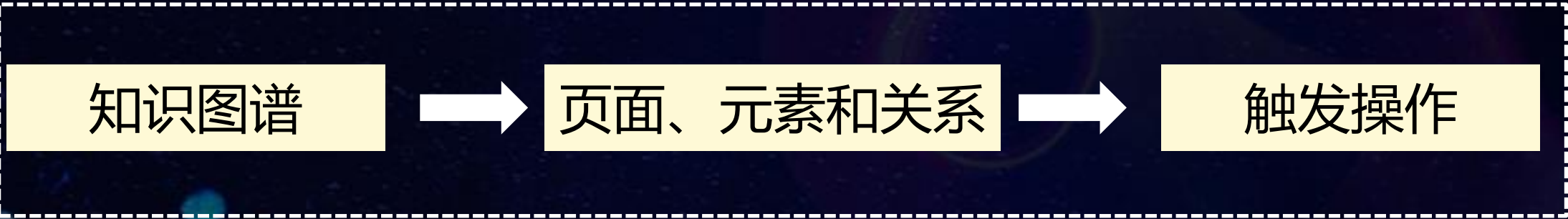
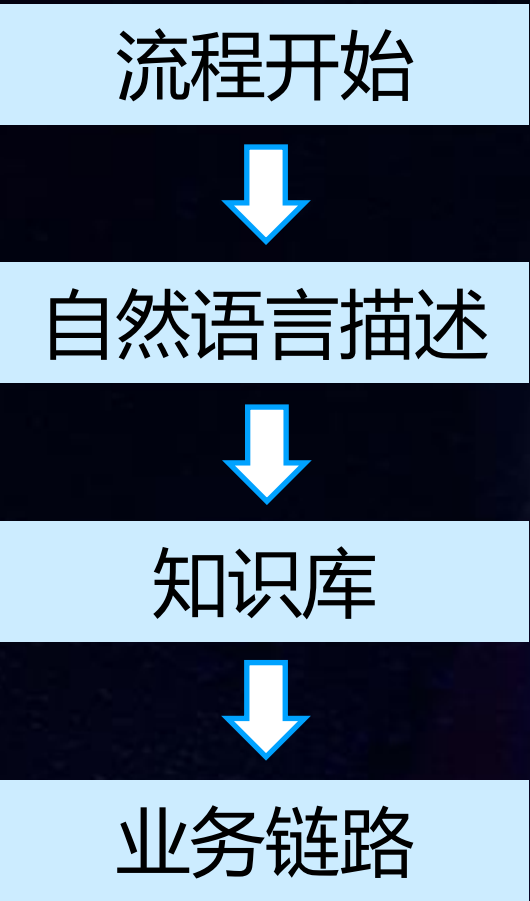
效率提升

AI执行自动化  
56%  
效率提升

寻找链路  
47%  
范围缩小

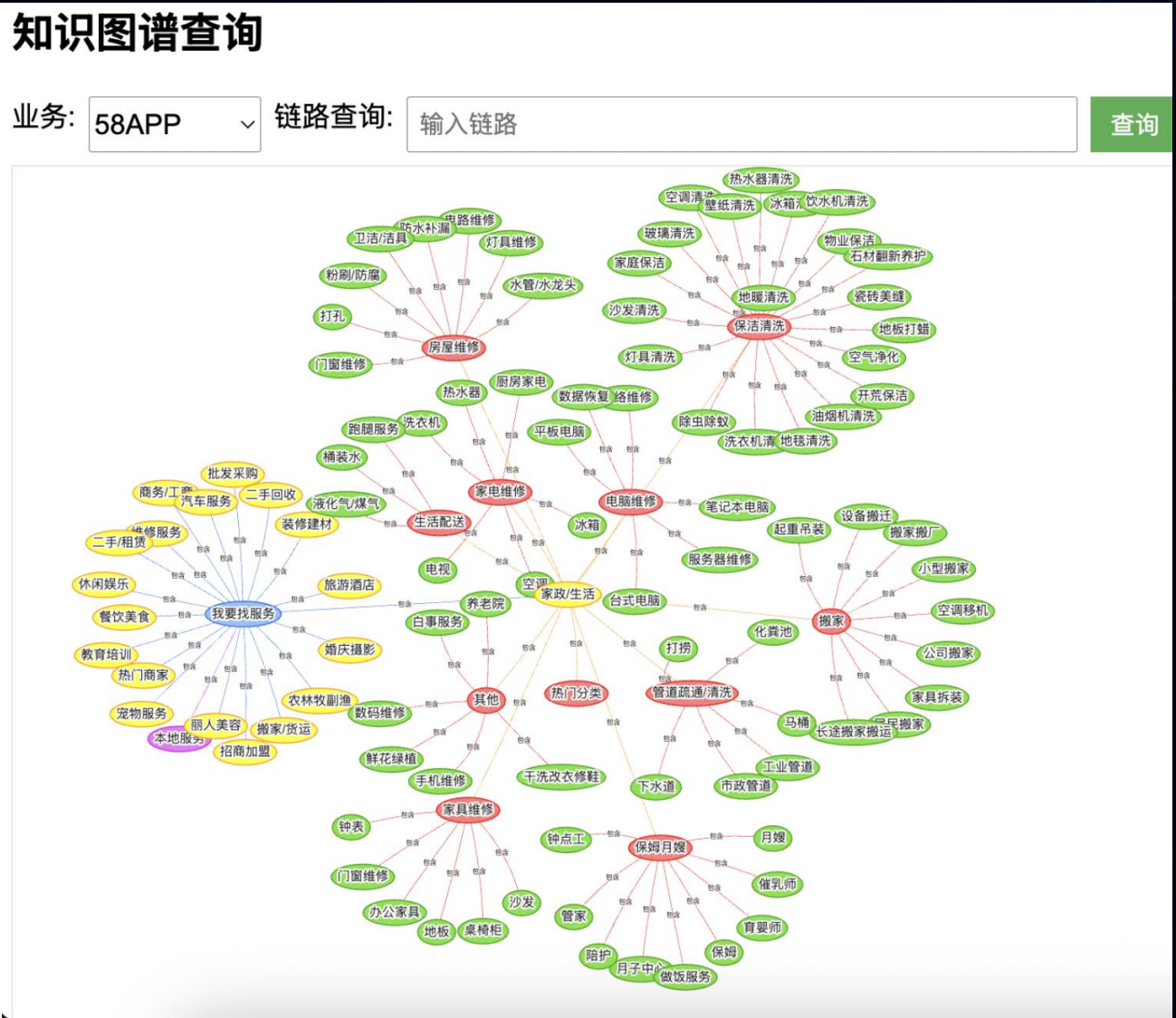


# 知识库融合知识图谱整体流程



流程结束

## 每一步与知识图谱提供的路径对比



若找不到图谱路径  
则大模型自行决策

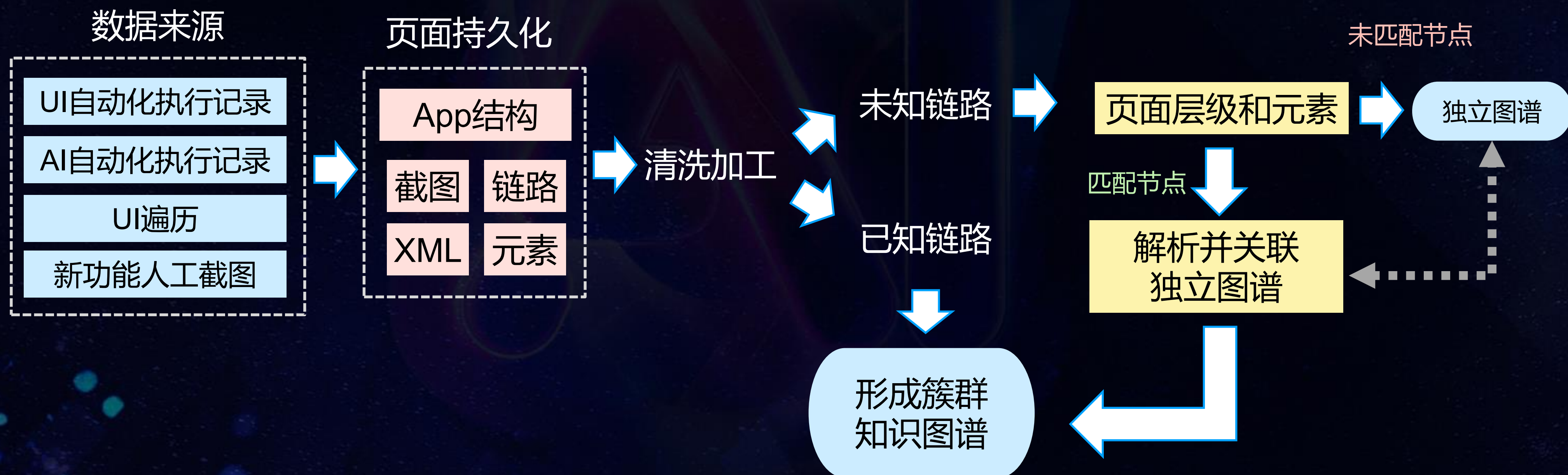
自动收集  
**60%**  
图谱链路覆盖



降低业务链路维护成本

## 如何自更新业务链路知识库

通过主动和被动的页面数据，按照页面和元素的包含关系自动更新单个节点



核心实现思路

- 1.ADB 获取当前 UI 结构：使用adb dump命令获取当前页面布局，解析出可交互控件。
- 2.MCP 协议执行操作：通过 MCP 协议将控件操作发送给设备执行。
- 3.页面状态识别：提取页面特征（如 Activity 名称、关键控件），识别页面跳转。
- 4.路径记录：记录每一步操作和跳转，形成路径图



# App截图更新到知识图谱

App 截图怎么使用:

- 元素类型: 弹窗、菜单、输入框、按钮、文本、选择器
- 元素间关系: 包含、依赖、触发、组合 等
- 元素属性: 文案、尺寸、层级、状态等



当前层级(0)OCR识别结果已存储, 共51个元素  
 点击元素边界坐标: [1005, 2341, 1071, 2377]  
 点击位置坐标: (1038, 2359)  
 点击元素文本: 我的  
 置信度: 0.9877171865149954

-----  
 OCR识别到56个文本元素  
 文本: 46, 置信度: 0.99  
 文本: 2:11, 置信度: 1.00  
 文本: Ks, 置信度: 1.00  
 文本: 99, 置信度: 1.00  
 文本: 附近, 置信度: 1.00  
 文本: 直播, 置信度: 0.99

```

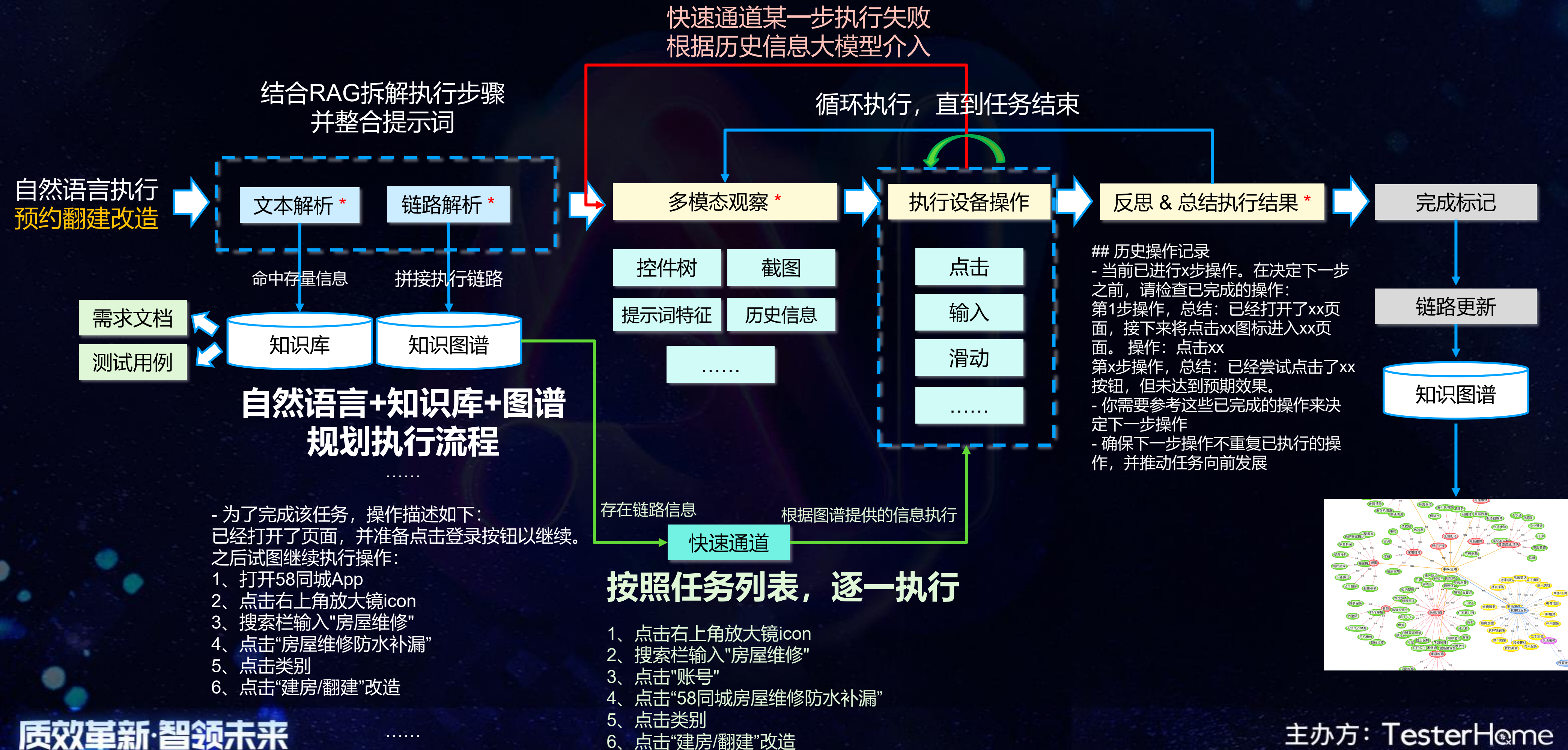
/* 处理结构关系 */
// 处理Top Navigation Bar
MERGE (parent:Component {name: "Top Navigation Bar"})
ON CREATE SET parent.id = apoc.create.uuid(),
    parent.type = "Top Navigation Bar",
    parent.position = "屏幕顶部固定",
    parent.content = "北京, 附近, 直播"
SET parent += apoc.map.clean(properties(parent), ["id", "name"], [])

// 处理Search Bar及其子组件
MERGE (searchBar:Component {name: "Search Bar"})
ON CREATE SET searchBar.id = apoc.create.uuid(),
    searchBar.type = "Search Bar",
    searchBar.position = "搜索栏",
    searchBar.content = "家装服务"
SET searchBar += apoc.map.clean(properties(searchBar), ["id", "name"], [])

MERGE (searchIcon:Component {name: "搜索图标按钮"})
ON CREATE SET searchIcon.id = apoc.create.uuid(),
    searchIcon.type = "图标按钮",
    searchIcon.content = "🔍"
  
```

# 整体执行流程

## 根据未完成部分重新规划执行流程



## 落地到业务

AI视觉  
大模型能力  
驱动UI自动化

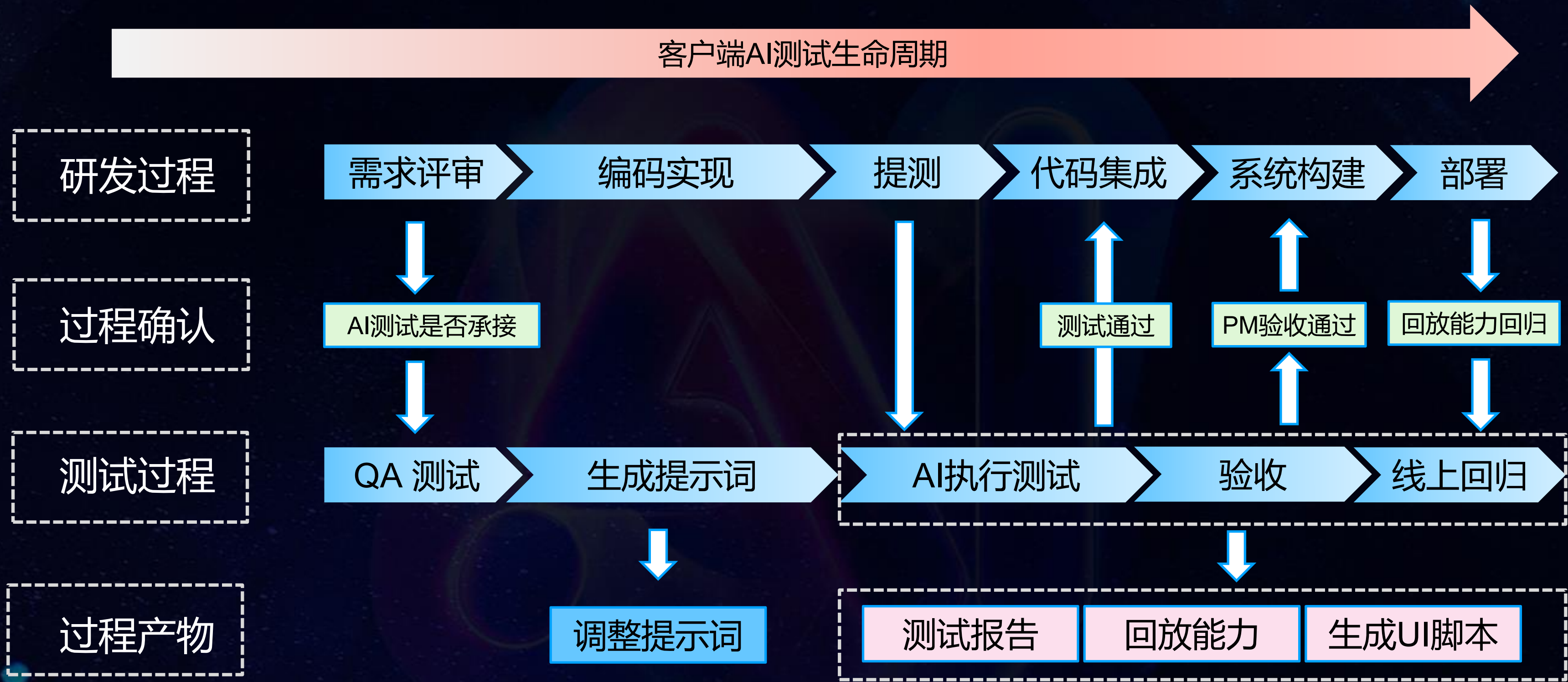
通过需求文档  
生成AI可执行的  
提示词

知识内容沉淀  
图谱补强链路  
准确性

落地业务  
产生价值

# 寻找边界、优化边界、突破边界





评审阶段筛选出AI测试需求，AI测试完成后提供测试报告，PM验收线上AI回归

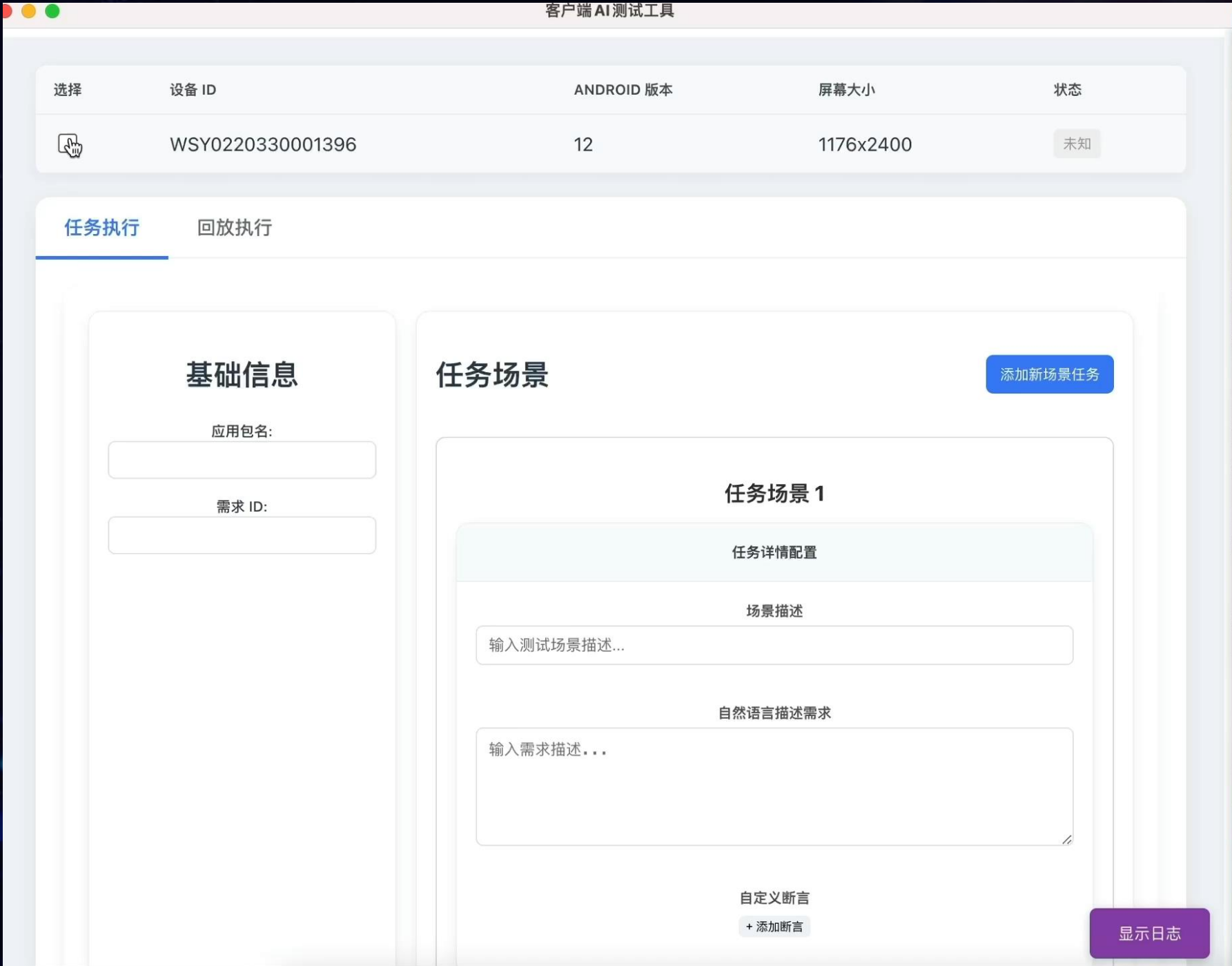
应用效果与展示

Q1  
9.6%  
AI测试完成率

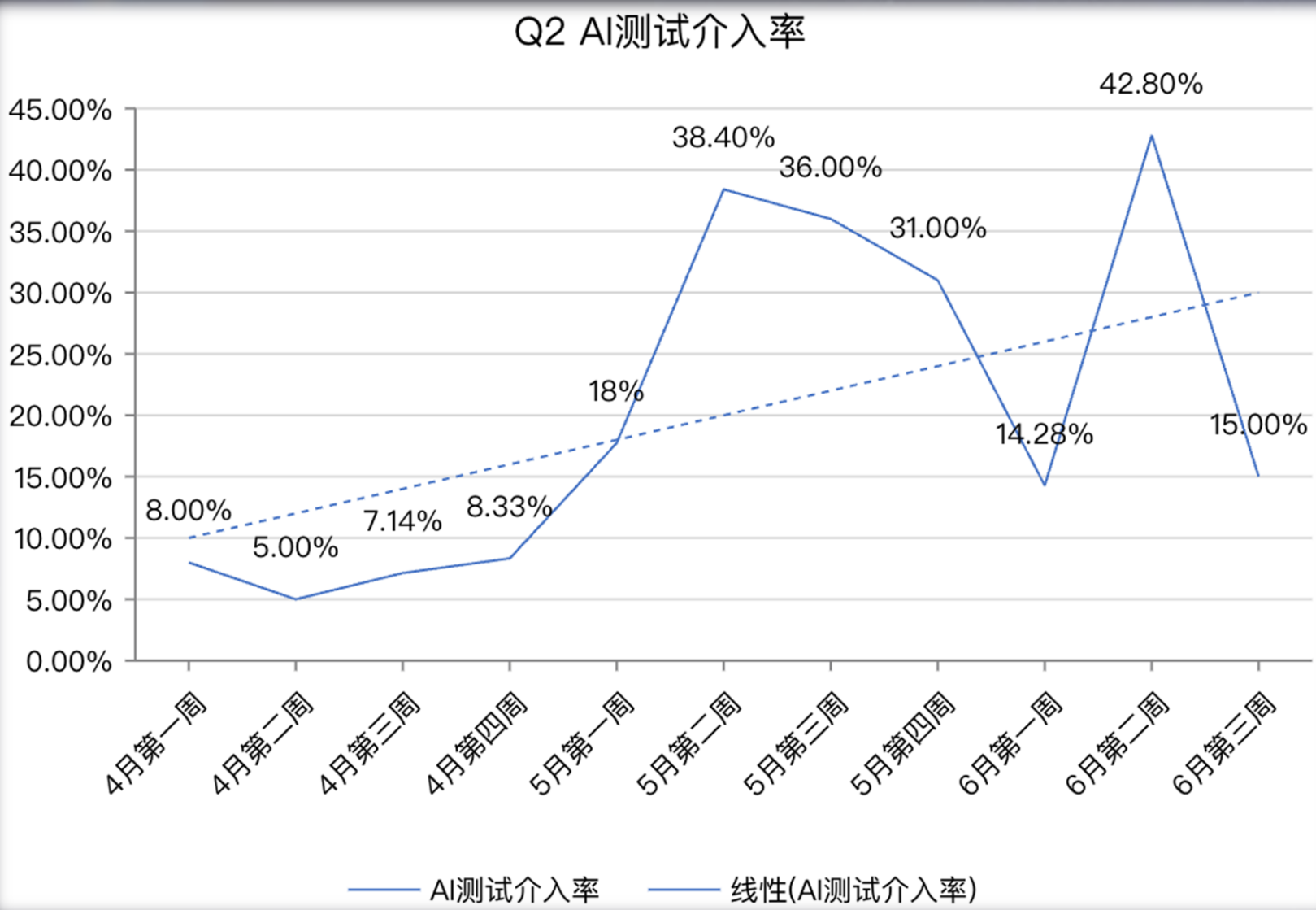
Q2  
23.52%  
AI测试完成率

26.06%  
人员工具  
使用覆盖度

43.7%  
知识库+图谱  
准确率提升



Q2 周度介入率



## 技术实现维度 (聚焦工具、能力建设)

- **架构迁移:**
  - 所有决策层迁移到云端，桌面工具只做执行
- **工具 / 能力封装:**
  - 封装 MCP
- **AI 能力赋能:**
  - 将 AI 能力赋能给 UI 自动化巡检和 RPA 方向
- **数据 / 链路优化:**
  - 根据执行链路结合提示词动态造数

## 业务价值维度 (关注测试流程、效率与质量)

- **测试流程完善:**
  - 增加测试未覆盖部分重测机制
- **测试协同推进:**
  - 同步进行 API 场景测试
- **测试目标达成:**
  - 提高需求覆盖度及测试效率
- **AI自查需求:**
  - 根据测试能力自动从需求list里给出可以AI测试的建议，人工复查

# 谢谢

THANKS



扫一扫上面的二维码图案，加我为朋友。